During OO design, you identify external things and make them classes in your code. But there's another source of classes in your designs. There are the classes that correspond to things inside your code itself. For example, we know that the program we're writing will need to consolidate and summarize CSV data feeds. But that's a very passive statement. Let's turn it into a design by asking ourselves *what* does the summarizing and consolidating. And the answer (in our case) is a *CSV reader*. Let's make it into a class as follows:

```
class CsvReader
  def initialize
    # ...
  end

  def read_in_csv_data(csv_file_name)
    # ...
  end

  def total_value_in_stock
    # ...
  end

  def number_of_each_isbn
    # ...
  end
end
```

We'd call it using something like this:

```
reader = CsvReader.new
reader.read_in_csv_data("file1.csv")
reader.read_in_csv_data("file2.csv")
  :         :              :
puts "Total value in stock = #{reader.total_value_in_stock}"
```

We need to be able to handle multiple CSV files, so our `reader` object needs to accumulate the values from each CSV file it is fed. We'll do that by keeping an array of values in an instance variable. And how shall we represent each book's data? Well, we just finished writing the `BookInStock` class, so that problem is solved. The only other question is how we parse data in a CSV file. Fortunately, Ruby comes with a good CSV library, which we will cover in detail in *Chapter 28, Library Reference: I/O*. Given a CSV file with a header line, we can iterate over the remaining rows and extract values by name:

tut_classes/stock_stats/csv_reader.rb

```
class CsvReader
  def initialize
    @books_in_stock = []
  end

  def read_in_csv_data(csv_file_name)
    CSV.foreach(csv_file_name, headers: true) do |row|
      @books_in_stock << BookInStock.new(row["ISBN"], row["Price"])
    end
  end
end
```

Because you're probably wondering what's going on, let's dissect that `read_in_csv_data` method. On the first line, we tell the CSV library to open the file with the given name. The `headers: true` option tells the library to parse the first line of the file as the names of the columns, and